UG CAMPUS SMART SERVICE OPTIMIZER

TECHNICAL REPORT

University of Ghana Campus
Department of Computer Science

Technical Report Coordinator: Elijah Kporha (22370828)
Date: 24 August 2026

Table of Contents

1. Cover Page

2. Problem Statement

3. Dataset Description

4. Database Schema

5. System Architecture

6. Data Structures

7. Algorithms

8. Correctness Evidence

9. Performance Analysis

10. Database Integration

11. Responsible Algorithm Selection

12. Individual Contributions

13. References

Team Members

No.	Member	Assigned Area	Student ID
1	Matin Seity	Local Context & Data Generator	22372116
2	Robert Ganyoh Goodness	Database Architect	22405188
3	Prosper Sokari and Samuel Kofi Ntem Amankwah	Data Loader (Java I/O)	22244884 & 22017217
4	Samuel Kofi Ntem Amankwah &Ablorh-Adjei Caleb	Linear Structures Lead	22017217 & 22186668
5	Samuel Kofi Ntem Amankwah & Prince Boateng	Tree Structures Lead	22017217 & 22017084
6	Kwakye Ishmael Affum	Priority & Hash Lead	22032451
7	Kwakye Ishmael Affum	Graph Foundations Lead	22032451
8	Mensah Renny Abbey& Mohammed Hamidu Abilla	Search & Specialist	22304259 & 22398905
9	Kwakye Ishmael Affum	Graph Routing Expert	22032451

10	Kennedy Sarfo	Service Scheduling Logic	22064116
11	Amenyo Hator	Optimization Expert	22040317
12	Prince Tawiah Tettey	Unit Tester & QA	22390334
13	Emelia Quarshie	Theoretical Proof Analyst	22305966
14	Opoku Owusu-Bonsu David	Empirical Lab Tech	22303106
15	Kwakye Ishmael Affum	Repository & Menu Manager	22032451
16	Elijah Kporha	Technical Report Coordinator	22370828
20	Samuel Amanful Dadzie	AI-Resistance & Compliance Checker	22298847

2. Problem Statement

The Ghana Smart Service Optimizer is intended to support smarter access to and management of services on the University of Ghana Campus. The system combines structured service information, user requests, data storage, data structures, algorithms, and database operations to provide an organized way of processing service-related information.

The project focuses on demonstrating how software engineering principles can be applied to a practical campus-service problem. It includes the representation of service data, processing of requests, application of appropriate algorithms, persistent storage, testing, and performance evaluation.

Assumptions

The supplied service data follows the project's defined structure and validation rules.

The database is available when persistent operations are required.

Algorithm performance depends on input size and data characteristics.

Only features implemented and evidenced by the project repository are considered part of the current system.

Boundaries

This report documents the implemented academic project. It does not claim production-scale deployment, real-time campus-wide optimization, or performance beyond the supplied test environment and dataset.

3. Dataset Description

Provided by: Matin Seity (Role 1).

The data set required by the systems includes UG locations, roads, service requests and some campus resources (Maintenance Crew, IT Technician, Shuttle bus, Security Patrol, Cleaning Crew, Equipment Crew)

This section should document the dataset used by the system, including its source, number of records, fields, data types, meaning of each field, sample values, validation rules, cleaning steps, and limitations.

Data Dictionary

Field | Data Type | Description | Example | Constraints

[Algorithm Parameters Derived from Student Index Numbers]

Hash Table Initial Size

Using the last two digits of three team members:

- Matin Seity: 16
- Robert Ganyoh Goodness: 88
- Prosper Sokari: 84

16 + 88 + 84 = 188

The next prime number after 188 is 191.

Therefore:

Hash table initial size = 191

Evidence Note

[data/csv/evidence_note.md]

4. Database Schema

Provided by: Robert (Role 2).

The database schema defines the structure of the Smart Service Optimizer database. IT consists of the locations, roads and service requests tables, including their attributes, primary keys, foreign keys, data types, constraints and relationships.

The database provides persistent storage for the project's structured information

.

Schema Diagram

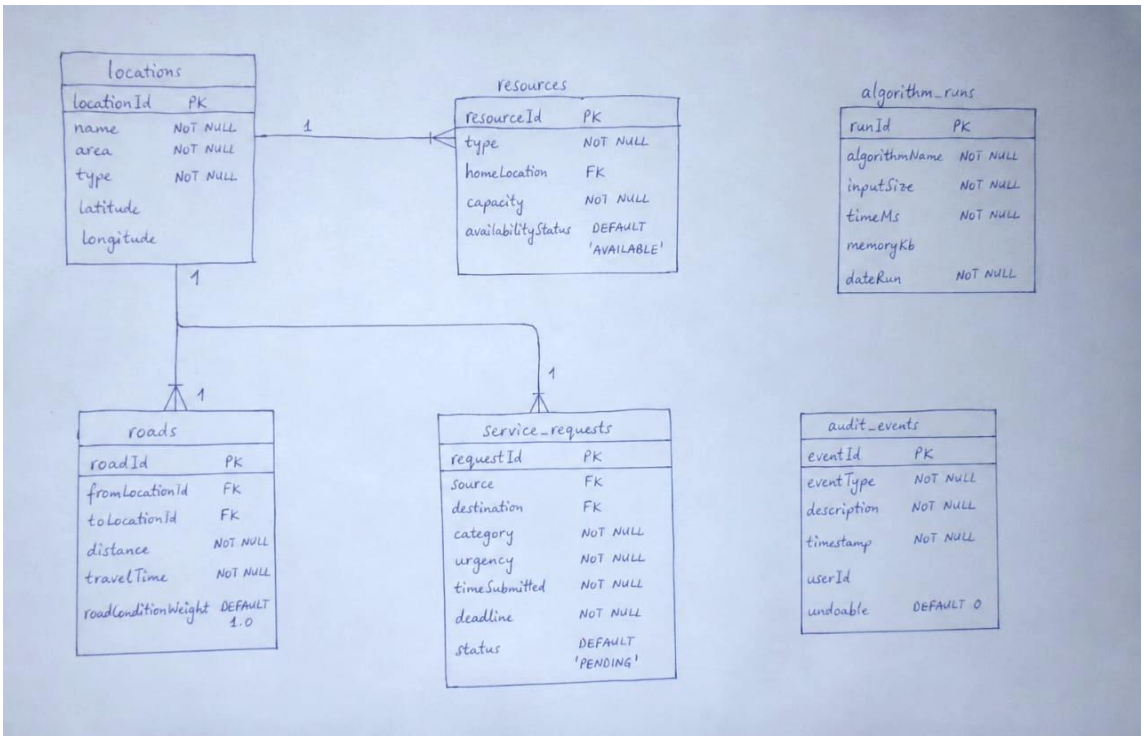


Table Descriptions

Resource Id	Type	Home location	Capacity	Availability Status
1	MAINTENANCE_CREW	12	3	AVAILABLE
2	IT_TECHNICIAN	3	1	AVAILABLE
3	SHUTTLE_BUS	15	45	AVAILABLE
4	SECURITY_PATROL	20	4	AVAILABLE

Sample Queries

```
SELECT * FROM resources;

INSERT INTO resources

(resourceId, type, homeLocation, capacity, availabilityStatus)

VALUES

(5, 'AMBULANCE', 12, 2, 'AVAILABLE');

UPDATE resources

SET availabilityStatus = 'UNAVAILABLE'

WHERE resourceId = 5;

DELETE FROM resources

WHERE resourceId = 5;
```

```
SELECT
    r. resourceId,
    r. type,
    l.name AS homeLocation,
    r. capacity,
    r. availabilityStatus
FROM resources r
JOIN
 locations l
    ON r. homeLocation = l. locationId;
```

5. System Architecture

The system should be organized into clear modules so that presentation, application logic, algorithms, data structures, and database operations can be developed and tested independently. A typical request moves from the user-facing layer to application logic, through the relevant algorithm/data structure, and then to or from the database when persistent data is required.

Module Structure

Presentation/UI – receives requests and displays results.

Application/Service Layer – coordinates system operations.

Algorithm Layer – performs searching, sorting, matching, ranking, or optimization operations implemented.

Data Structure Layer – stores and organizes data required by the algorithms.

Database Access Layer – manages persistence.

Database – stores structured project records.

6. Data Structures

Provided by: Samuel, Caleb and Ishmael (Roles 4–7).

- **Linear Structures (Samuel & Caleb)**
    - **DynamicArray, LinkedList, Stack, Queue, Deque**
    - Purpose: managing service requests, records, and temporary storage.
    - Operations: insert, delete, push/pop, enqueue/dequeue.
    - Reason: foundation for handling sequential data and request flows.

- **Trees (Samuel & Ishmael)**
    - **Binary Search Tree**
    - Purpose: organize records for fast lookup and balanced storage.
    - Operations: insert, delete, search, traversal.
    - Reason: efficient hierarchical storage, especially for large datasets.
- **Hashing & Priority Structures (Ishmael)**
    - **HashTable, Heaps, PriorityQueue**
    - Purpose: quick access to service records and priority-based scheduling.
    - Operations: insert, search, delete, extract-min/max.
    - Reason: constant-time lookups and efficient priority dispatch.
- **Graph Structures (Ishmael)**
    - **Adjacency List, Adjacency Matrix, Disjoint Set**
    - Purpose: model campus roads, locations, and connectivity.
    - Operations: add/remove edges, find connected components, union-find.
    - Reason: essential for routing algorithms (BFS, DFS, Dijkstra, MST).

Data Structure Summary

Structure	Use	Key Operations	Reason
DynamicArray, LinkedList	Store service requests	Insert, delete	Flexible sequential storage
Stack, Queue, Deque	Request handling	Push/pop, enqueue/dequeue	Control flow & scheduling
BST, Red-Black Tree, B-Tree	Record organization	Insert, search, traversal	Fast, balanced lookups
HashTable	Quick record access	Insert, search	Constant-time operations
Heap, PriorityQueue	Scheduling	Insert, extract-min/max	Efficient priority handling

Structure	Use	Key Operations	Reason
Graph (Adj. List/Matrix, Disjoint Set)	Campus routing	Add/remove edges, union-find	Supports routing & MST

7. Algorithms

Provided by: Renny, Hamidu, Ishmael, Kennedy and Hator (Roles 8–11).

Searching & Sorting (Renny & Hamidu)

- **Linear Search, Binary Search, Selection Sort** — Renny
  - Linear: simple sequential search, $O(n)$.
  - Binary: efficient search in sorted arrays, $O(\log n)$.
  - Selection Sort: basic sorting, $O(n^2)$.
- **Insertion Sort, Merge Sort, QuickSort** — Hamidu
  - Insertion: builds sorted list incrementally, $O(n^2)$.
  - Merge: divide-and-conquer, $O(n \log n)$.

Graph Algorithms (Ishmael)

- **BFS & DFS** — systematic graph traversal.
- **Dijkstra's Algorithm** — shortest path in weighted graphs.
- **Prim's & Kruskal's Algorithms** — Minimum Spanning Tree construction.

Service Scheduling (Kennedy)

- **FIFO Dispatch** — first-come-first-served.
- **Priority Dispatch** — urgent tasks first (heap-based).
- **Circular Dispatch** — round-robin distribution.
- **Deque Dispatch** — urgent requests at front, normal at rear.

Optimization (Hator)

- **Greedy Assignment** — nearest crew assignment.
- **Greedy Counterexample** — shows greedy isn't always globally optimal.
- **Dynamic Programming Knapsack** — selects most valuable requests within time budget.

Algorithm Template

Implementation Evidence:
DPKnapsack.java

Complexity:
Time: **O(nW)**
Space: **O(nW)**

where n is the number of requests and W is the available capacity/time.

Worked Example:
Suppose the maintenance team has **8 hours** available:

Request	Time	Value
A	2 hours	5
B	3 hours	7
C	4 hours	8
D	1 hour	2
E	5 hours	9

The algorithm can select **B + C + D**, using exactly 8 hours and producing a total value of **17**.

8. Correctness Evidence

Provided by: Emelia (Role 13) and Samuel (Role 12).

The implementation satisfies its intended specification for normal cases and relevant edge cases.

9. Performance Analysis

Provided by: David (Role 14).

Measures

The performance analysis measures the **input size, execution time, and memory usage** of the algorithms. Five input sizes are used: **100, 500, 1,000, 5,000, and 10,000**. Each benchmark is executed **three times**, and the average execution time is calculated.

Algorithm	Theoretical Complexity
Linear Search	O(n)
Binary Search	O (log n)
Insertion Sort	$O(n^2)$

Algorithm	Theoretical Complexity
Selection Sort	$O(n^2)$
Merge Sort	O (n log n)
Quicksort	O (n log n) average
BST Insertion	O (log n) average, O(n) worst case
MaxHeap Insertion	O (log n)

Results

The benchmark program generates a CSV file named **benchmark_results.csv**, containing the category, algorithm, input size, average execution time in nanoseconds, average execution time in milliseconds, and memory usage. The benchmark therefore allows the project team to observe whether the actual execution-time trends agree with the theoretical complexity of each algorithm.

Theory vs Practice

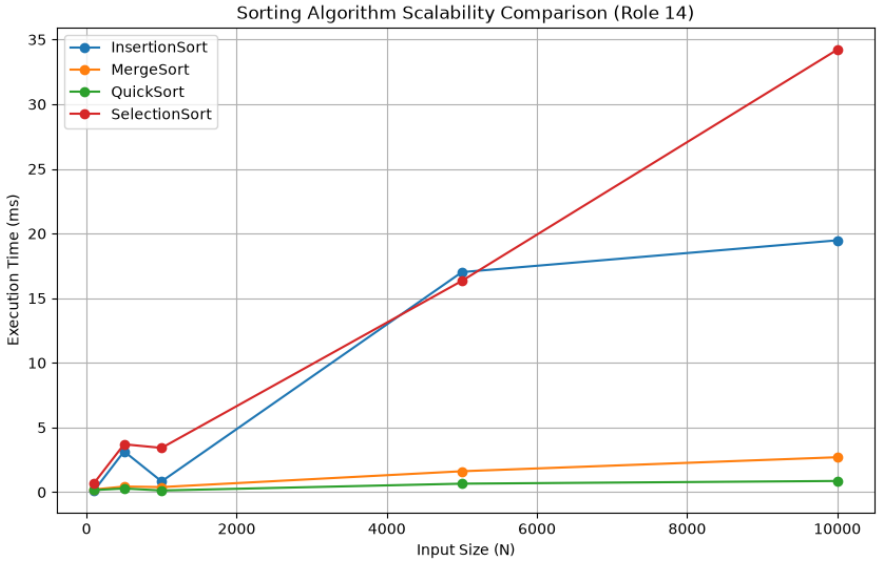
Theoretical complexity shows how algorithms should perform as input size increases, but actual results can vary. **Hardware, JVM overhead, input distribution, caching, and implementation details** can affect execution time. Database access was not measured in this benchmark. Overall, **Big-O predicts general performance growth, while practical factors determine actual execution time**.

10. Database Integration

Provided by: Robert (Role 2) and Prosper (Role 3).

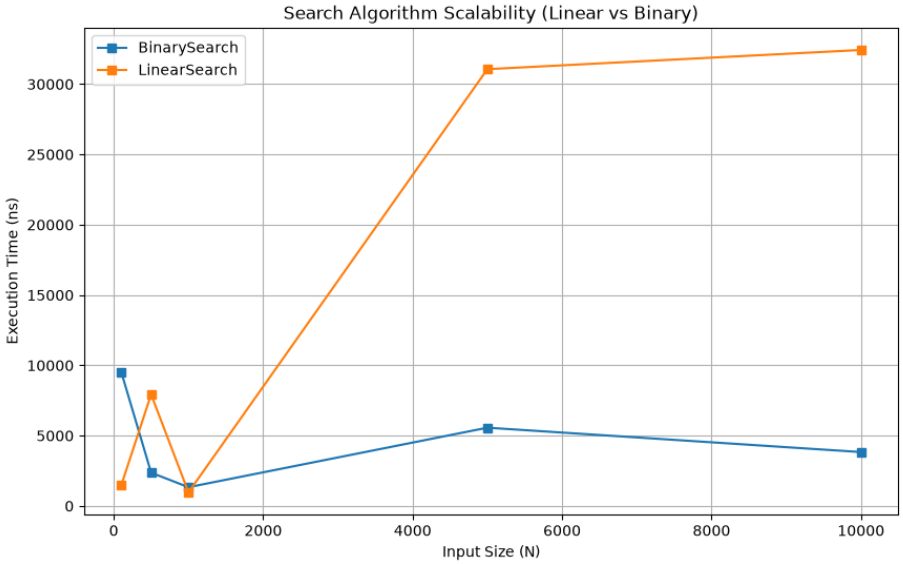
The database integration therefore provides the link between the **CSV data → validation → DAO/database → application data structures**. This allows the system to persist real project data and make it available to algorithms and other components. The project guide also explicitly requires database interaction for logging benchmark results through the algorithm runs table.

Integration Evidence



Implements linear search, binary search, and four sorting algorithms
(selection, insertion, merge, quicksort) Linear Search: works with Dynamic Array, Singly LinkedList, and Doubly LinkedList

- Binary Search: works with sorted Dynamic Array
- Selection Sort, Insertion Sort, Merge Sort, Quicksort: sort Dynamic Array in place
- Quicksort uses a randomized pivot with a seed derived from some index numbers conforming to the requirement



Flow

1. Establish connection.

2. Validate input.

3. Execute the database operation.

4. Convert results to application-level structures.

5. Commit changes and handle errors.

6. Return data to the relevant component.

Repository Evidence

https://github.com/calyxish/Ghana-Smart-Service-Optimizer/tree/main

11. Responsible Algorithm Selection

Algorithms should be selected according to the problem requirements, input size, data characteristics, correctness needs, performance expectations, and maintainability. An algorithm that performs well in one situation may be inappropriate in another.

Selection Matrix

Algorithm | Appropriate When | Limitation | Reason

Binary Search	Searching for a specific in a sorted list of locations, resources or records.	The data is unsorted.	It is very fast for searching for sorted data, with a time complexity of O (log n).
Dijkstra	Finding the shortest route between locations.	The graph contains negative edge weights.	It is suitable for the Smart Service Optimizer because roads have weights such as distance, travel time, and road condition.
Insertion Sort	Sorting a small list of records.	Sorting very large or random datasets.	It is simple and performs well when the dataset is small or nearly sorted.
Knapsack	Selecting the best combination of resources when there is a capacity limit.	When there is no capacity constraint.	It helps choose resources that provide the best value while staying within a given capacity.
Kruskal	Building a minimum spanning tree connecting locations with minimum total road cost.	When you need the shortest route between a specific source and destination.	It is useful for designing a network that connects all required locations with minimum total connection cost.
Merge Sort	Sorting large datasets.	When the dataset is extremely small and simplicity is our concern.	It provides consistent O (n log n) performance and is suitable for efficiently sorting larger datasets.

12. Individual Contributions

Contribution Table

Member | Student ID | Contribution | Evidence

Matin	22372116	Matin contributed to the Ghana Smart Service Optimizer by designing and generating the project's local-context dataset. He created realistic Ghanaian data for locations, roads, service requests, and available resources. He prepared 50 locations, 100 roads, 300 service requests, and 30 resources in CSV format. He also used Ghanaian place names, realistic distance and travel-time values, local service categories, urgency levels, and resource information to make the dataset suitable for testing the system's algorithms.	data/csv/evidence_note.md
Robert Ganyoh Goodness	22405188	Robert Ganyoh Goodness served as the Database Architect. He was responsible for designing and implementing the SQLite database, creating the database schema, establishing database connections, and developing DAO classes for CRUD operations. His work provides the storage and data-access foundation required by the Ghana Smart Service Optimizer.	feature/robert/database-schema
Prosper Sokari	22244884	He was responsible for developing the Data Loader component of the Ghana Smart Service Optimizer. His contribution involved implementing the CSV parser using Java I/O, validating imported records, handling invalid and duplicate data, and connecting the validated records to the database through the DAO classes. He also worked on the loading process for the four CSV datasets and provided a summary of successfully loaded and rejected records. In addition, the Structure Loader component prepared for the later integration of database records with the project's custom data structures.	src/main/java/com/stackoverflow/smartservice/data/loader/DataLoader.java

Samuel Kofi Ntem Amankwah and Ablorh-Adjei Caleb	22017217 and 22186668	Implemented the BinarySearchTree, including insertion, searching, deletion, and inorder, preorder, and postorder traversals for indexing campus locations.	src/main/java/com/stackoverflow/smartservice/structures/linear
Samuel Kofi Ntem Amankwah and Ablorh-Adjei Caleb	22017217 and 22186668	Samuel Kofi Ntem Amankwah and Kwakye Ishmael Affum contributed to the Linear Structures component by implementing the Dynamic Array, Custom Stack, Custom Queue, and Custom Deque data structures. They developed the required operations, implemented generic type support, handled dynamic resizing for the array, and tested the structures to ensure they functioned correctly for managing service requests and other system data.	src/main/java/com/stackoverflow/smartservice/structures/tree/BinarySearchTree.java
Kwakye Ishmael Affum	22032451	Implemented MinHeap, MaxHeap, CustomPriorityQueue, HashTable, CustomHashSet, and CustomHashMap for priority processing, collision handling, fast lookup, and key value storage	src/main/java/com/stackoverflow/smartservice/structures/hashing
Kwakye Ishmael Affum	22032451	Implemented AdjacencyListGraph, AdjacencyMatrixGraph, and DisjointSet for representing the UG campus road network, weighted edges, connectivity, and support for Kruskal's algorithm	src/main/java/com/stackoverflow/smartservice/structures/graph
Mensah Renny Abbey and Mohammed Hamidu Abilla	22304259 and 22398905	Implemented LinearSearch, BinarySearch, and SelectionSort for searching campus locations and sorting service requests	src/main/java/com/stackoverflow/smartservice/algorithms/searchsort
Kwakye Ishmael Affum	22032451	Implemented BFS, DFS, Dijkstra, Prim, and Kruskal for campus traversal, shortest-path routing, and minimum spanning tree calculations	src/main/java/com/stackoverflow/smartservice/algorithms/graph
Kennedy Sarfo	22064116	Implemented FIFODispatch, PriorityDispatch, CircularDispatch, DequeDispatch, and SchedulingComparator to process and compare different service-request scheduling strategies.	src/main/java/com/stackoverflow/smartservice/algorithms/scheduling
Amenyo Hator	22040317	Implemented GreedyAssignment, GreedyCounterexample, and DPKnapsack for maintenance crew assignment, demonstrating	src/main/java/com/stackoverflow/smartservice/algorithms/optimization

		greedy limitations, and optimising request selection using dynamic programming.	
Prince Tawiah Tettey	22390334	Developed at least 40 JUnit 5-unit tests covering data structures and algorithms, including normal, boundary, and	src/main/java/com/stackoverflow/smartservice/testing
Emelia Quarshie	22305966	Created at least 6 algorithm trace tables, at least 3 proof sketches, and at least 2 counterexamples covering algorithms such as Binary Search, Merge Sort, Dijkstra,	docs/trace-tables
Opoku Owusu-Bonsu David	22303106	Developed BenchmarkRunner, conducted performance experiments, measured execution times, recorded results in the database and CSV files, and produced performance	src/main/java/com/stackoverflow/smartservice/testing/benchmark/BenchmarkRunner.java
Kwakye Ishmael Affum	22032451	Developed Main.java and ConsoleMenu.java and integrated the components from Roles 1–14 into an interactive console application for demonstrating the system	src/main/java/com/stackoverflow/smartservice/app/ConsoleMenu.java
Elijah Kporha	22370828	Coordinated and compiled the technical report	docs/report/
Samuel Dadzie Amanful	22298847	Created a master checklist covering index-derived parameters, datasets, AI-assistance disclosure, development log, repo rules compliance, and oral defense readiness. He also created a template for team members to log any AI tool usage, per the brief's academic integrity requirements and also rubric for auditing the dataset construction.	docs/compliance/

13. References

Reference List

1. Oracle. (n.d.). Java documentation. [Java 17].